

AWS Red Team vs Blue Team Cybersecurity Homelab

Complete Setup Guide for Linux (Parrot OS)

Table of Contents

- [1. Project Overview](#)
- [2. Network Topology](#)
- [3. Prerequisites](#)
- [4. Part 1: AWS Environment Setup](#)
- [5. Part 2: VPC and Network Configuration](#)
- [6. Part 3: EC2 Instance Deployment](#)
- [7. Part 4: Instance Access Configuration](#)
- [8. Part 5: Security Tools Installation](#)
- [9. Part 6: Attack Simulation & Testing](#)
- [10. Part 7: Cost Management & Cleanup](#)
- [11. Resources & References](#)

Project Overview

Description

This project creates a cloud-based penetration testing and security monitoring lab using AWS infrastructure. The lab simulates a **Red Team vs Blue Team** environment where you can practice offensive security techniques (penetration testing) while simultaneously monitoring and defending against attacks using industry-standard SIEM and monitoring tools.

Architecture Components

- Kali Linux Attacker Box** (Red Team): Offensive security platform for penetration testing
- Windows Server Target**: Vulnerable target system for testing attacks
- Ubuntu Security Tools Box** (Blue Team): Defensive monitoring with Splunk and Nessus
- Custom VPC**: Isolated network environment for secure testing
- Traffic Mirroring**: Optional advanced monitoring of network packets

Skills Demonstrated

- AWS cloud infrastructure design and deployment
- Virtual Private Cloud (VPC) configuration
- Security group and network ACL management
- Linux system administration and CLI operations
- Penetration testing with Kali Linux tools
- SIEM implementation and log analysis (Splunk)
- Vulnerability assessment (Nessus)
- Network security monitoring
- Incident detection and response

Estimated Costs

- Daily Cost**: \$5-15 USD (depending on instance types)
- Cost Optimization**: Stop instances when not in use to minimize charges
- Free Tier Eligible**: Some components may qualify for AWS Free Tier

Time Requirements

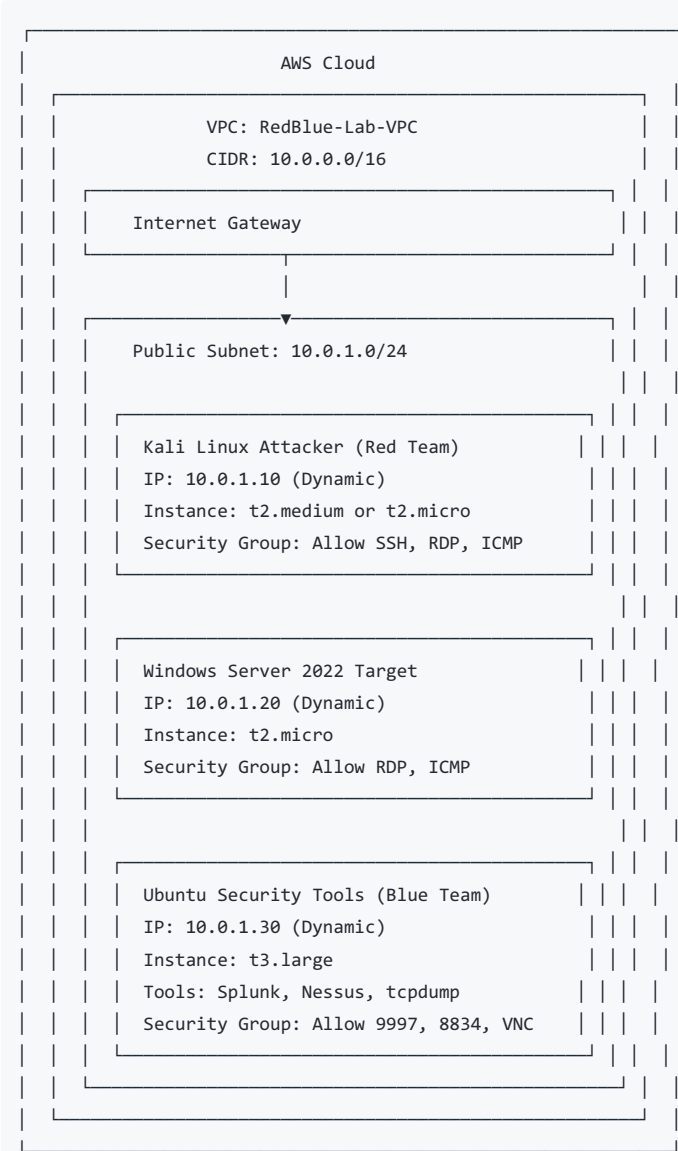
- Initial Setup**: 4-6 hours
- Tool Configuration**: 2-3 hours
- Total**: 6-9 hours for complete deployment

Network Topology

VPC Design

AWS Region: us-east-2 (Ohio) or your preferred region
VPC CIDR: 10.0.0.0/16
Public Subnet CIDR: 10.0.1.0/24

Instance Layout



Network Traffic Flow

1. **Internet Gateway** provides public internet access
2. **Public Subnet** hosts all three EC2 instances
3. **Security Groups** act as virtual firewalls controlling traffic
4. **Traffic Mirroring** (optional) duplicates packets to Security Tools box
5. **Splunk Forwarder** sends Windows logs from target to monitoring box

Prerequisites

AWS Account Requirements

- Active AWS account with billing enabled
- IAM user with programmatic access
- Credit card on file (for billing)
- Awareness of AWS pricing models

Local System Requirements (Parrot OS/Linux)

Operating System: Parrot OS, Kali Linux, Ubuntu 20.04+, or any Debian-based distribution

Installed Tools:

```
# Check if tools are installed
aws --version          # AWS CLI
ssh -V                 # OpenSSH client
remmina --version      # RDP client (optional)
git --version          # Git (for cloning repos)
unzip --version        # Unzip utility
curl --version         # cURL for downloads
```

Required Software Installation (Parrot OS)

1. Install AWS CLI

```
# Download AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

# Unzip the installer
unzip awscliv2.zip

# Run installer
sudo ./aws/install

# Verify installation
aws --version
# Expected output: aws-cli/2.x.x Python/3.x.x Linux/x.x.x
```

2. Install Remmina RDP Client (for Windows access)

```
# Update package repository
sudo apt update

# Install Remmina with RDP plugin
sudo apt install -y remmina remmina-plugin-rdp remmina-plugin-vnc

# Verify installation
remmina --version
```

Alternative installation methods:

```
# Using Flatpak (system-agnostic)
flatpak remote-add --user --if-not-exists flathub https://flathub.org/repo/flathub.flatpakrepo
flatpak install --user flathub org.remmina.Remmina
flatpak run --user org.remmina.Remmina

# Using Snap
sudo snap install remmina
```

3. Install Additional Tools

```
# Install text editors
sudo apt install -y nano vim

# Install network tools
sudo apt install -y net-tools tcpdump nmap

# Install SSH tools (usually pre-installed)
sudo apt install -y openssh-client

# Install unzip if not present
sudo apt install -y unzip curl wget
```

AWS Knowledge Prerequisites

- Basic understanding of AWS EC2 instances
- Familiarity with VPC concepts (subnets, security groups)
- Understanding of SSH key pairs and RDP
- Basic Linux command line skills
- Understanding of networking fundamentals (IP addressing, ports, protocols)

Part 1: AWS Environment Setup

Step 1.1: Create IAM User for CLI Access

Why: You need programmatic access credentials to interact with AWS via CLI.

Using AWS Console:

1. **Log into AWS Console**
 - Navigate to: <https://console.aws.amazon.com/>
 - Sign in with your root account or IAM user
2. **Access IAM Service**
 - In the search bar at top, type `IAM`
 - Click on "IAM" service
3. **Create New User**
 - Click "Users" in left sidebar
 - Click "Create user" button
 - **Username:** `aws-redblue-lab-user`
 - Click "Next"
4. **Set Permissions**
 - Select "Attach policies directly"
 - Search and attach these policies:
 - ✓ **AmazonEC2FullAccess**
 - ✓ **AmazonVPCFullAccess**
 - Click "Next"
 - Review and click "Create user"
5. **Generate Access Keys**
 - Click on the newly created user: `aws-redblue-lab-user`
 - Go to "Security credentials" tab
 - Scroll to "Access keys" section
 - Click "Create access key"
 - Select use case: "Command Line Interface (CLI)"
 - Check acknowledgment box
 - Click "Next"
 - Add description tag (optional): `RedBlue Lab CLI Access`
 - Click "Create access key"
6. **Save Credentials Securely**

```
Access Key ID: AKIA*****
Secret Access Key: wJalr*****/*****
```

⚠ **CRITICAL:** Download the CSV file or copy both keys immediately. You won't see the secret key again!

Step 1.2: Configure AWS CLI on Parrot OS

Configure Default Profile

```
# Open terminal on Parrot OS
# Run AWS configure command
aws configure

# You'll be prompted for the following:
AWS Access Key ID [None]: AKIA*****      # Paste your access key
AWS Secret Access Key [None]: wJalr***** # Paste your secret key
Default region name [None]: us-east-2      # Choose region (Ohio)
Default output format [None]: json         # Use JSON format
```

Verify Configuration

```
# Check AWS CLI configuration
aws configure list

# Expected output:
#      Name                Value                Type    Location
#      ----                -
#      profile              <not set>           None    None
#      access_key           *****LBSW shared-credentials-file
#      secret_key           *****QKwi shared-credentials-file
#      region               us-east-2           config-file    ~/.aws/config

# Test connection by listing EC2 instances
aws ec2 describe-instances --region us-east-2

# If successful, you'll see JSON output (empty if no instances exist)
```

Configuration Files Location

```
# AWS CLI stores credentials in your home directory
ls -la ~/.aws/

# You should see:
# ~/.aws/config      - Contains region and output format
# ~/.aws/credentials - Contains access keys (NEVER share this!)

# View config file
cat ~/.aws/config
# Output:
# [default]
# region = us-east-2
# output = json

# View credentials (censored for security)
cat ~/.aws/credentials
# Output:
# [default]
# aws_access_key_id = AKIA*****
# aws_secret_access_key = wJalr*****
```

Step 1.3: Create EC2 Key Pair

Purpose: SSH key pairs are required to securely connect to Linux instances (Kali, Ubuntu).

Method 1: Using AWS CLI (Recommended for Linux)

```
# Create key pair and save to ~/.ssh directory
aws ec2 create-key-pair \
  --key-name redblue-lab-keypair \
  --query 'KeyMaterial' \
  --output text > ~/.ssh/redblue-lab-keypair.pem

# Set proper permissions (required for SSH)
chmod 400 ~/.ssh/redblue-lab-keypair.pem

# Verify key exists
ls -l ~/.ssh/redblue-lab-keypair.pem
# Output: -r----- 1 user user 1674 Nov 06 14:30 /home/user/.ssh/redblue-lab-keypair.pem

# Verify in AWS that key was created
aws ec2 describe-key-pairs --key-names redblue-lab-keypair
```

Method 2: Using AWS Console

1. Navigate to EC2 Dashboard
2. Click "Key Pairs" under "Network & Security" in left sidebar
3. Click "Create key pair"
4. **Name:** redblue-lab-keypair
5. **Key pair type:** RSA
6. **Private key file format:** .pem (for Linux/Mac)
7. Click "Create key pair"
8. The .pem file will download automatically
9. Move it to your SSH directory:

```
mv ~/Downloads/redblue-lab-keypair.pem ~/.ssh/  
chmod 400 ~/.ssh/redblue-lab-keypair.pem
```

Part 2: VPC and Network Configuration

Step 2.1: Create Custom VPC

Purpose: Isolated network environment for your security lab.

Using AWS CLI:

```
# Create VPC with 10.0.0.0/16 CIDR block  
VPC_ID=$(aws ec2 create-vpc \  
  --cidr-block 10.0.0.0/16 \  
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=RedBlue-Lab-VPC}]' \  
  --query 'Vpc.VpcId' \  
  --output text)  
  
echo "VPC Created: $VPC_ID"  
  
# Enable DNS hostnames (required for instances to get public DNS)  
aws ec2 modify-vpc-attribute \  
  --vpc-id $VPC_ID \  
  --enable-dns-hostnames  
  
# Enable DNS resolution  
aws ec2 modify-vpc-attribute \  
  --vpc-id $VPC_ID \  
  --enable-dns-support  
  
# Verify VPC creation  
aws ec2 describe-vpcs --vpc-ids $VPC_ID
```

Using AWS Console:

1. Navigate to **VPC Dashboard**
2. Click "Create VPC" button
3. Select **"VPC and more"** (creates VPC with necessary components)
4. **Configuration:**
 - Name tag: RedBlue-Lab-VPC
 - IPv4 CIDR block: 10.0.0.0/16
 - IPv6 CIDR block: No IPv6
 - Tenancy: Default
 - Number of Availability Zones: 1
 - Number of public subnets: 1
 - Number of private subnets: 0
 - NAT gateways: None
 - VPC endpoints: None
 - DNS options: ✓ Enable DNS hostnames, ✓ Enable DNS resolution
5. Review the preview diagram on right side
6. Click "Create VPC"
7. Wait for creation to complete (1-2 minutes)

Step 2.2: Create Public Subnet

Note: If you used "VPC and more", the subnet is already created. Skip to verification.

Using AWS CLI:

```
# Create public subnet
SUBNET_ID=$(aws ec2 create-subnet \
  --vpc-id $VPC_ID \
  --cidr-block 10.0.1.0/24 \
  --availability-zone us-east-2a \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=RedBlue-Public-Subnet}]' \
  --query 'Subnet.SubnetId' \
  --output text)

echo "Subnet Created: $SUBNET_ID"

# Enable auto-assign public IP for instances in this subnet
aws ec2 modify-subnet-attribute \
  --subnet-id $SUBNET_ID \
  --map-public-ip-on-launch

# Verify subnet
aws ec2 describe-subnets --subnet-ids $SUBNET_ID
```

Step 2.3: Create and Attach Internet Gateway

Using AWS CLI:

```
# Create Internet Gateway
IGW_ID=$(aws ec2 create-internet-gateway \
  --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=RedBlue-IGW}]' \
  --query 'InternetGateway.InternetGatewayId' \
  --output text)

echo "Internet Gateway Created: $IGW_ID"

# Attach Internet Gateway to VPC
aws ec2 attach-internet-gateway \
  --internet-gateway-id $IGW_ID \
  --vpc-id $VPC_ID

# Verify attachment
aws ec2 describe-internet-gateways --internet-gateway-ids $IGW_ID
```

Step 2.4: Configure Route Table

Using AWS CLI:

```

# Get the main route table ID for the VPC
ROUTE_TABLE_ID=$(aws ec2 describe-route-tables \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=association.main,Values=true" \
  --query 'RouteTables[0].RouteTableId' \
  --output text)

echo "Route Table ID: $ROUTE_TABLE_ID"

# Add route to Internet Gateway (0.0.0.0/0 -> IGW)
aws ec2 create-route \
  --route-table-id $ROUTE_TABLE_ID \
  --destination-cidr-block 0.0.0.0/0 \
  --gateway-id $IGW_ID

# Tag the route table
aws ec2 create-tags \
  --resources $ROUTE_TABLE_ID \
  --tags Key=Name,Value=RedBlue-Public-RouteTable

# Verify route
aws ec2 describe-route-tables --route-table-ids $ROUTE_TABLE_ID

```

Step 2.5: Create Security Groups

Security Groups act as virtual firewalls for your instances.

Security Group 1: Kali Linux Attacker & Windows Target


```

# Create security group for Kali and Windows
SG_MAIN=$(aws ec2 create-security-group \
    --group-name RedBlue-Main-SG \
    --description "Security group for Kali attacker and Windows target" \
    --vpc-id $VPC_ID \
    --query 'GroupId' \
    --output text)

echo "Main Security Group Created: $SG_MAIN"

# Add inbound rules

# SSH access (port 22) from your IP
MY_IP=$(curl -s https://checkip.amazonaws.com)
aws ec2 authorize-security-group-ingress \
    --group-id $SG_MAIN \
    --protocol tcp \
    --port 22 \
    --cidr ${MY_IP}/32

# RDP access (port 3389) from your IP
aws ec2 authorize-security-group-ingress \
    --group-id $SG_MAIN \
    --protocol tcp \
    --port 3389 \
    --cidr ${MY_IP}/32

# ICMP (ping) from anywhere within VPC
aws ec2 authorize-security-group-ingress \
    --group-id $SG_MAIN \
    --protocol icmp \
    --port -1 \
    --cidr 10.0.0.0/16

# All traffic from within security group (instance-to-instance)
aws ec2 authorize-security-group-ingress \
    --group-id $SG_MAIN \
    --source-group $SG_MAIN \
    --protocol all

# Verify rules
aws ec2 describe-security-groups --group-ids $SG_MAIN

```

Security Group 2: Ubuntu Security Tools Box

```

# Create security group for Security Tools box
SG_SECURITY_TOOLS=$(aws ec2 create-security-group \
    --group-name RedBlue-SecurityTools-SG \
    --description "Security group for Ubuntu Security Tools monitoring box" \
    --vpc-id $VPC_ID \
    --query 'GroupId' \
    --output text)

echo "Security Tools Group Created: $SG_SECURITY_TOOLS"

# SSH access from your IP
aws ec2 authorize-security-group-ingress \
    --group-id $SG_SECURITY_TOOLS \
    --protocol tcp \
    --port 22 \
    --cidr ${MY_IP}/32

# VNC access (port 5900) from your IP (for Netspectrum Ubuntu Desktop)
aws ec2 authorize-security-group-ingress \

```

```
--group-id $SG_SECURITY_TOOLS \  
--protocol tcp \  
--port 5900 \  
--cidr ${MY_IP}/32  
  
# HTTP for VNC web interface (varies by setup)  
aws ec2 authorize-security-group-ingress \  
  --group-id $SG_SECURITY_TOOLS \  
  --protocol tcp \  
  --port 80 \  
  --cidr ${MY_IP}/32  
  
# HTTPS for VNC web interface  
aws ec2 authorize-security-group-ingress \  
  --group-id $SG_SECURITY_TOOLS \  
  --protocol tcp \  
  --port 443 \  
  --cidr ${MY_IP}/32  
  
# Splunk web interface (port 8000)  
aws ec2 authorize-security-group-ingress \  
  --group-id $SG_SECURITY_TOOLS \  
  --protocol tcp \  
  --port 8000 \  
  --cidr ${MY_IP}/32  
  
# Splunk receiver port (9997) from VPC  
aws ec2 authorize-security-group-ingress \  
  --group-id $SG_SECURITY_TOOLS \  
  --protocol tcp \  
  --port 9997 \  
  --cidr 10.0.0.0/16  
  
# Nessus web interface (port 8834)  
aws ec2 authorize-security-group-ingress \  
  --group-id $SG_SECURITY_TOOLS \  
  --protocol tcp \  
  --port 8834 \  
  --cidr ${MY_IP}/32  
  
# ICMP from VPC  
aws ec2 authorize-security-group-ingress \  
  --group-id $SG_SECURITY_TOOLS \  
  --protocol icmp \  
  --port -1 \  
  --cidr 10.0.0.0/16  
  
# All traffic from main security group  
aws ec2 authorize-security-group-ingress \  
  --group-id $SG_SECURITY_TOOLS \  
  --source-group $SG_MAIN \  
  --protocol all
```

Part 3: EC2 Instance Deployment

Step 3.1: Find Latest AMI IDs

AMIs (Amazon Machine Images) are templates for EC2 instances.

Find Kali Linux AMI

```
# Search for official Kali Linux AMI
aws ec2 describe-images \
  --owners 679593333241 \
  --filters "Name=name,Values=kali-linux-*" "Name=state,Values=available" \
  --query 'sort_by(Images, &CreationDate)[-1].[ImageId,Name,Description]' \
  --output table

# Example output:
# |  ami-0a1b2c3d4e5f6g7h8  |  kali-linux-2023.4-amd64  |  Official Kali Linux  |

# Save the AMI ID
KALI_AMI="ami-0a1b2c3d4e5f6g7h8" # Replace with actual ID from output
```

Find Windows Server AMI

```
# Search for Windows Server 2022 Base AMI
aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=Windows_Server-2022-English-Full-Base-*" "Name=state,Values=available" \
  --query 'sort_by(Images, &CreationDate)[-1].[ImageId,Name]' \
  --output table

# Save the AMI ID
WINDOWS_AMI="ami-0123456789abcdef0" # Replace with actual ID
```

Find Ubuntu Desktop AMI (Netspectrum)

```
# For Netspectrum Ubuntu Desktop, you'll need to subscribe via AWS Marketplace
# Search AWS Marketplace: https://aws.amazon.com/marketplace
# Search for "Netspectrum Ubuntu Desktop"
# Subscribe (it's free) and note the AMI ID

# Alternative: Use standard Ubuntu Server and install GUI
UBUNTU_AMI=$(aws ec2 describe-images \
  --owners 099720109477 \
  --filters "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*" "Name=state,Values=available" \
  --query 'sort_by(Images, &CreationDate)[-1].ImageId' \
  --output text)

echo "Ubuntu AMI: $UBUNTU_AMI"
```

Step 3.2: Launch Kali Linux Attacker Instance

```

# Launch Kali Linux instance
KALI_INSTANCE=$(aws ec2 run-instances \
  --image-id $KALI_AMI \
  --instance-type t2.medium \
  --key-name redblue-lab-keypair \
  --security-group-ids $SG_MAIN \
  --subnet-id $SUBNET_ID \
  --associate-public-ip-address \
  --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":20,"VolumeType":"gp3"}}]' \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Kali-Attacker},{Key=Role,Value=RedTeam}]' \
  --query 'Instances[0].InstanceId' \
  --output text)

echo "Kali Instance Launched: $KALI_INSTANCE"

# Wait for instance to be running
aws ec2 wait instance-running --instance-ids $KALI_INSTANCE
echo "Kali instance is now running"

# Get public IP
KALI_IP=$(aws ec2 describe-instances \
  --instance-ids $KALI_INSTANCE \
  --query 'Reservations[0].Instances[0].PublicIpAddress' \
  --output text)

echo "Kali Public IP: $KALI_IP"

```

Step 3.3: Launch Windows Server Target Instance

```

# Launch Windows Server instance
WINDOWS_INSTANCE=$(aws ec2 run-instances \
  --image-id $WINDOWS_AMI \
  --instance-type t2.micro \
  --key-name redblue-lab-keypair \
  --security-group-ids $SG_MAIN \
  --subnet-id $SUBNET_ID \
  --associate-public-ip-address \
  --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":30,"VolumeType":"gp3"}}]' \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Windows-Target},{Key=Role,Value=Target}]' \
  --query 'Instances[0].InstanceId' \
  --output text)

echo "Windows Instance Launched: $WINDOWS_INSTANCE"

# Wait for instance to be running
aws ec2 wait instance-running --instance-ids $WINDOWS_INSTANCE

# Get public IP
WINDOWS_IP=$(aws ec2 describe-instances \
  --instance-ids $WINDOWS_INSTANCE \
  --query 'Reservations[0].Instances[0].PublicIpAddress' \
  --output text)

echo "Windows Public IP: $WINDOWS_IP"

```

Step 3.4: Launch Ubuntu Security Tools Instance

```

# Launch Ubuntu Security Tools instance
SECURITY_INSTANCE=$(aws ec2 run-instances \
  --image-id $UBUNTU_AMI \
  --instance-type t3.large \
  --key-name redblue-lab-keypair \
  --security-group-ids $SG_SECURITY_TOOLS \
  --subnet-id $SUBNET_ID \
  --associate-public-ip-address \
  --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":50,"VolumeType":"gp3"}}]' \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Security-Tools},{Key=Role,Value=BlueTeam}]' \
  --query 'Instances[0].InstanceId' \
  --output text)

echo "Security Tools Instance Launched: $SECURITY_INSTANCE"

# Wait for instance to be running
aws ec2 wait instance-running --instance-ids $SECURITY_INSTANCE

# Get public IP
SECURITY_IP=$(aws ec2 describe-instances \
  --instance-ids $SECURITY_INSTANCE \
  --query 'Reservations[0].Instances[0].PublicIpAddress' \
  --output text)

echo "Security Tools Public IP: $SECURITY_IP"

```

Step 3.5: Save Instance Information

```
# Create a reference file for easy access
cat > ~/redblue-lab-instances.txt << EOF
=====

AWS RED TEAM vs BLUE TEAM LAB - INSTANCES
=====

VPC ID: $VPC_ID
Subnet ID: $SUBNET_ID
Main Security Group: $SG_MAIN
Security Tools SG: $SG_SECURITY_TOOLS

-----
KALI LINUX ATTACKER (Red Team)
-----
Instance ID: $KALI_INSTANCE
Public IP: $KALI_IP
Private IP: $(aws ec2 describe-instances --instance-ids $KALI_INSTANCE --query 'Reservations[0].Instances[0].PrivateIpAddress' --out
SSH Command: ssh -i ~/.ssh/redblue-lab-keypair.pem kali@$KALI_IP

-----
WINDOWS SERVER TARGET
-----
Instance ID: $WINDOWS_INSTANCE
Public IP: $WINDOWS_IP
Private IP: $(aws ec2 describe-instances --instance-ids $WINDOWS_INSTANCE --query 'Reservations[0].Instances[0].PrivateIpAddress' --
RDP: Use Remmina with IP $WINDOWS_IP
Username: Administrator

-----
UBUNTU SECURITY TOOLS (Blue Team)
-----
Instance ID: $SECURITY_INSTANCE
Public IP: $SECURITY_IP
Private IP: $(aws ec2 describe-instances --instance-ids $SECURITY_INSTANCE --query 'Reservations[0].Instances[0].PrivateIpAddress' -
SSH Command: ssh -i ~/.ssh/redblue-lab-keypair.pem ubuntu@$SECURITY_IP

=====
Created: $(date)
=====
EOF

cat ~/redblue-lab-instances.txt
```

Part 4: Instance Access Configuration

Step 4.1: Access Kali Linux via SSH

Initial SSH Connection:

```
# SSH into Kali Linux (default user is 'kali')
ssh -i ~/.ssh/redblue-lab-keypair.pem kali@$KALI_IP

# If you get "connection refused", wait 2-3 minutes for instance to fully boot
# First time connection will ask to accept host key - type 'yes'

# Once connected, you'll see Kali Linux prompt:
# └─(kali@ip-10-0-1-x)-[~]
# └─$
```

Install XRDP for GUI Access (Optional):

```
# Create installation script
cat > install-xrdp.sh << 'EOF'
#!/bin/bash

echo "[*] Updating system packages..."
sudo apt-get update
sudo apt-get full-upgrade -y

echo "[*] Installing Xfce4 desktop and XRDP..."
sudo apt-get install -y kali-desktop-xfce xorg xrdp

echo "[*] Configuring XRDP..."
sudo sed -i 's/port=3389/port=3389/g' /etc/xrdp/xrdp.ini

echo "[*] Starting XRDP service..."
sudo systemctl enable xrdp
sudo systemctl start xrdp

echo "[*] XRDP installation complete!"
echo "[*] You can now RDP to this instance on port 3389"
EOF

# Make script executable
chmod +x install-xrdp.sh

# Run installation (this takes 15-30 minutes)
./install-xrdp.sh

# Set Kali user password for RDP login
sudo passwd kali
# Enter new password when prompted

# Exit SSH session
exit
```

Connect via RDP (using Remmina):

```
# Launch Remmina on your Parrot OS
remmina &

# In Remmina:
# 1. Click "+" to create new connection
# 2. Protocol: RDP
# 3. Server: [paste Kali public IP]
# 4. Username: kali
# 5. Password: [password you just set]
# 6. Resolution: Use client resolution
# 7. Click "Save and Connect"
```

Step 4.2: Access Windows Server via RDP

Retrieve Windows Administrator Password:

```
# Get encrypted password from AWS
aws ec2 get-password-data \
  --instance-id $WINDOWS_INSTANCE \
  --priv-launch-key ~/.ssh/redblue-lab-keypair.pem \
  --query 'PasswordData' \
  --output text

# This command decrypts and displays the Administrator password
# Copy this password - you'll need it for RDP
```

Alternative Method (Manual Decryption):

```
# Get encrypted password
ENCRYPTED_PASSWORD=$(aws ec2 get-password-data \
  --instance-id $WINDOWS_INSTANCE \
  --query 'PasswordData' \
  --output text)

# Save to file
echo "$ENCRYPTED_PASSWORD" | base64 -d > /tmp/encrypted_password.bin

# Decrypt using private key
openssl rsautl -decrypt -in /tmp/encrypted_password.bin \
  -inkey ~/.ssh/redblue-lab-keypair.pem

# Clean up
rm /tmp/encrypted_password.bin
```

Connect via Remmina:

```
# Launch Remmina
remmina &

# Configuration:
# Protocol: RDP
# Server: [Windows public IP]
# Username: Administrator
# Password: [decrypted password from above]
# Domain: (leave empty)
# Resolution: Use client resolution
# Color depth: True color (24 bpp)
# Save and Connect
```

Disable Windows Firewall (Required for Splunk):

Once connected to Windows:

1. Click Start menu
2. Type "Windows Defender Firewall"
3. Click "Turn Windows Defender Firewall on or off"
4. Select "Turn off" for both:
 - Private network settings
 - Public network settings
5. Click OK

Step 4.3: Access Ubuntu Security Tools Box

For Standard Ubuntu Server (if using custom Ubuntu AMI):


```
# SSH into Ubuntu
ssh -i ~/.ssh/redblue-lab-keypair.pem ubuntu@$SECURITY_IP

# Install desktop environment and VNC server
sudo apt update
sudo apt upgrade -y

# Install Ubuntu Desktop
sudo apt install -y ubuntu-desktop

# Install TigerVNC Server
sudo apt install -y tigervnc-standalone-server tigervnc-common

# Set VNC password
vncpasswd
# Enter password when prompted (min 6 characters)
# Choose 'n' for view-only password

# Start VNC server
vncserver :1 -geometry 1920x1080 -depth 24

# Configure VNC to start on boot
sudo systemctl enable vncserver@:1.service
sudo systemctl start vncserver@:1.service
```

For Netspectrum Ubuntu Desktop:

```
# Access via web browser
# Open browser and navigate to:
https://$SECURITY_IP

# Or use the public IP directly:
https://[Security-Tools-Public-IP]

# Initial password is the EC2 Instance ID
# Get instance ID:
aws ec2 describe-instances \
  --instance-ids $SECURITY_INSTANCE \
  --query 'Reservations[0].Instances[0].InstanceId' \
  --output text

# Copy instance ID, paste as password
# Once logged in, change VNC password immediately
```

Connect via VNC (Remmina):

```
# Launch Remmina
remmina &

# Configuration:
# Protocol: VNC
# Server: [Security Tools public IP]:5901 # Note the :5901 for display :1
# Password: [VNC password you set]
# Color depth: True color (24 bit)
# Quality: Best
# Save and Connect
```

Part 5: Security Tools Installation

Step 5.1: Install Splunk Enterprise on Ubuntu Security Tools

Access Ubuntu Security Tools box (via VNC or SSH).

Download and Install Splunk:

```
# SSH method (or use terminal in VNC session)
ssh -i ~/.ssh/redblue-lab-keypair.pem ubuntu@$SECURITY_IP

# Download Splunk Enterprise (free trial)
cd /tmp
wget -O splunk-9.1.2-b6b9c8185839-linux-2.6-amd64.deb \
'https://download.splunk.com/products/splunk/releases/9.1.2/linux/splunk-9.1.2-b6b9c8185839-linux-2.6-amd64.deb'

# Install Splunk
sudo dpkg -i splunk-9.1.2-b6b9c8185839-linux-2.6-amd64.deb

# Change to Splunk directory
cd /opt/splunk/bin

# Start Splunk (first time setup)
sudo ./splunk start --accept-license

# You'll be prompted to create admin credentials:
# Administrator Username: admin
# Administrator Password: [choose strong password - min 8 characters]
```

Configure Splunk:

```
# Enable Splunk to start on boot
sudo /opt/splunk/bin/splunk enable boot-start -user splunk

# Access Splunk Web Interface
# Open browser (on Security Tools box or your local machine)
# Navigate to: http://[Security-Tools-Public-IP]:8000
# Login with admin credentials you just created
```

Create Splunk Receiving Port:

In Splunk Web Interface:

1. Settings → Forwarding and Receiving
2. Click "Configure receiving"
3. Click "New Receiving Port"
4. Port: 9997
5. Click "Save"

Create Custom Index for Windows Security Logs:

1. Settings → Indexes
2. Click "New Index"
3. Index Name: win-security
4. Leave other settings as default
5. Click "Save"

Step 5.2: Install Splunk Universal Forwarder on Windows Target

On Windows Server (via RDP):

1. Open Microsoft Edge browser
2. Navigate to: https://www.splunk.com/en_us/download/universal-forwarder.html
3. Sign in or create free account
4. Download: Universal Forwarder for Windows (64-bit .msi)
5. Run the downloaded splunkforwarder-9.1.2-x64-release.msi

Installation Wizard:

1. Welcome screen → Check "I accept the license agreement" → Next
2. Installation type → "Customize Options" → Next
3. User credentials:
 - Username: admin
 - Password: [create password]
 - Next
4. Deployment Server: (leave blank) → Next
5. Receiving Indexer:
 - Host or IP: [Private IP of Security Tools box - 10.0.1.x]
 - Port: 9997
 - Next
6. Installation path: (default C:\Program Files\SplunkUniversalForwarder) → Install
7. Wait for installation → Finish

Configure inputs.conf for Security Event Logs:

1. Open File Explorer
2. Navigate to: C:\Program Files\SplunkUniversalForwarder\etc\system\local\
3. Create new file: inputs.conf
4. Edit inputs.conf with Notepad
5. Add the following content:

```
[WinEventLog://Security]
index = win-security
disabled = 0
```

```
[WinEventLog://Application]
index = win-security
disabled = 0
```

```
[WinEventLog://System]
index = win-security
disabled = 0
```

6. Save and close

Restart Splunk Forwarder:

1. Open Command Prompt as Administrator
2. Run:

```
cd "C:\Program Files\SplunkUniversalForwarder\bin"
splunk.exe restart
```
3. Check service status:

```
sc query SplunkForwarder
```

Verify in Splunk:

1. Back in Splunk Web Interface (on Security Tools box)
2. Search bar → Enter: index=win-security
3. You should see Windows event logs appearing
4. If no data, wait 5 minutes and refresh

Step 5.3: Install Tenable Nessus on Ubuntu Security Tools

Download and Install Nessus Essentials:

```
# SSH into Security Tools box
ssh -i ~/.ssh/redblue-lab-keypair.pem ubuntu@$SECURITY_IP

# Download Nessus Essentials (free version)
cd /tmp
wget https://www.tenable.com/downloads/api/v1/public/pages/nessus/downloads/[VERSION]/Nessus-[VERSION]-debian10_amd64.deb

# Alternative: Use exact URL (check Tenable website for latest version)
# Example for Nessus 10.6.3:
wget https://www.tenable.com/downloads/api/v1/public/pages/nessus/downloads/24057/Nessus-10.6.3-debian10_amd64.deb

# Install Nessus
sudo dpkg -i Nessus-*.deb

# Start Nessus service
sudo systemctl start nessusd.service

# Enable Nessus to start on boot
sudo systemctl enable nessusd.service

# Check service status
sudo systemctl status nessusd.service
```

Initial Nessus Setup:

1. Open browser and navigate to: [https://\[Security-Tools-Public-IP\]:8834](https://[Security-Tools-Public-IP]:8834)
2. Accept the security certificate warning
3. Select "Nessus Essentials" (free for personal use)
4. Click "Continue"
5. Register for activation code:
 - Go to: <https://www.tenable.com/products/nessus/nessus-essentials>
 - Fill out registration form
 - Check email for activation code
6. Back in Nessus web interface:
 - Enter activation code
 - Click "Continue"
7. Create Nessus administrator account:
 - Username: admin
 - Password: [create strong password]
 - Click "Submit"
8. Wait for plugins to compile (15-30 minutes)
9. Once complete, you can create and run scans

Create First Scan:

1. In Nessus dashboard → Click "New Scan"
2. Select "Basic Network Scan"
3. Settings:
 - Name: Windows Target Scan
 - Targets: [Windows Private IP - 10.0.1.x]
 - Credentials: (optional - for authenticated scan)
4. Click "Save"
5. Click "Launch" to run scan
6. Wait for scan to complete (5-15 minutes)
7. Review vulnerabilities found

Step 5.4: Install Additional Tools on Kali Linux

SSH into Kali or use RDP session:

```
# Update Kali repositories
sudo apt update

# Install Metasploit Framework (usually pre-installed)
sudo apt install -y metasploit-framework

# Install Nmap (usually pre-installed)
sudo apt install -y nmap

# Install additional tools
sudo apt install -y \
    netcat \
    tcpdump \
    wireshark \
    john \
    hashcat \
    hydra \
    sqlmap \
    nikto \
    enum4linux \
    smbclient \
    crackmapexec

# Update Metasploit database
sudo msfdb init
sudo msfconsole -q
# Type 'exit' to quit
```

Part 6: Attack Simulation & Testing

Step 6.1: Network Reconnaissance

From Kali Linux, scan the Windows target:

```
# SSH into Kali
ssh -i ~/.ssh/redblue-lab-keypair.pem kali@$KALI_IP

# Get Windows private IP (replace with actual IP)
WINDOWS_PRIVATE_IP="10.0.1.20"

# Ping test
ping -c 4 $WINDOWS_PRIVATE_IP

# Basic Nmap scan
nmap $WINDOWS_PRIVATE_IP

# Detailed scan with service detection
nmap -sV -sC -p- $WINDOWS_PRIVATE_IP -oN windows_scan.txt

# OS detection
sudo nmap -O $WINDOWS_PRIVATE_IP

# Vulnerability scan
nmap --script vuln $WINDOWS_PRIVATE_IP
```

Expected Results:

PORT	STATE	SERVICE	VERSION
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
445/tcp	open	microsoft-ds	Windows Server 2022
3389/tcp	open	ms-wbt-server	Microsoft Terminal Services

Step 6.2: Metasploit Attack Simulation

Launch Metasploit:

```
# On Kali Linux
msfconsole

# You'll see Metasploit banner
```

Example: SMB Enumeration:

```
# In Metasploit console
use auxiliary/scanner/smb/smb_version
set RHOSTS 10.0.1.20
run

# SMB Enumeration for shares
use auxiliary/scanner/smb/smb_enumshares
set RHOSTS 10.0.1.20
run
```

Example: RDP Brute Force (Educational Purposes Only):

```
use auxiliary/scanner/rdp/rdp_scanner
set RHOSTS 10.0.1.20
run
```

⚠ IMPORTANT ETHICAL NOTICE:

- Only perform attacks on systems you own or have explicit permission to test
- This lab is for educational purposes
- Never use these techniques on systems you don't own

Step 6.3: Monitor Attacks in Splunk

Access Splunk Web Interface:

```
http://[Security-Tools-Public-IP]:8000
```

Search for Security Events:

```
# In Splunk search bar, run these queries:
```

```
# View all Windows security logs  
index=win-security
```

```
# Failed login attempts  
index=win-security EventCode=4625
```

```
# Successful logins  
index=win-security EventCode=4624
```

```
# Account lockouts  
index=win-security EventCode=4740
```

```
# Network connections  
index=win-security EventCode=5156
```

```
# Time range: Last 60 minutes
```

Create Real-Time Dashboard:

1. Run search: index=win-security EventCode=4625
2. Click "Save As" → Dashboard Panel
3. Dashboard Title: "Security Monitoring"
4. Panel Title: "Failed Login Attempts"
5. Panel Type: Statistics Table
6. Click "Save"

Step 6.4: Analyze with Nessus

Run Vulnerability Scan:

1. Access Nessus: [https://\[Security-Tools-Public-IP\]:8834](https://[Security-Tools-Public-IP]:8834)
2. My Scans → Windows Target Scan → Click Launch
3. Wait for scan to complete
4. Click on completed scan to view results

Review Findings:

Nessus categorizes vulnerabilities:

- Critical: Immediate action required
- High: Serious vulnerabilities
- Medium: Moderate risk
- Low: Minimal risk
- Info: Informational findings

Generate Report:

1. Click "Export" button
2. Format: PDF or HTML
3. Chapters: Select all
4. Click "Export"
5. Download and review report

Step 6.5: Network Traffic Capture

On Ubuntu Security Tools box:

```
# SSH into Security Tools box
ssh -i ~/.ssh/redblue-lab-keypair.pem ubuntu@$SECURITY_IP

# Capture traffic on primary network interface
sudo tcpdump -i eth0 -w /tmp/lab_traffic.pcap

# Filter for specific traffic (example: RDP)
sudo tcpdump -i eth0 port 3389 -w /tmp/rdp_traffic.pcap

# Filter for traffic from Windows target
sudo tcpdump -i eth0 host 10.0.1.20 -w /tmp/windows_traffic.pcap

# Stop capture: Ctrl+C

# Analyze capture file
tcpdump -r /tmp/lab_traffic.pcap

# Copy to local machine for Wireshark analysis
exit # Exit SSH
scp -i ~/.ssh/redblue-lab-keypair.pem ubuntu@$SECURITY_IP:/tmp/lab_traffic.pcap ~/
wireshark ~/lab_traffic.pcap
```

Part 7: Cost Management & Cleanup

Step 7.1: Stop Instances (Preserve Setup)

Stop all instances when not in use to minimize costs:

```
# Stop instances (you won't be charged for compute, only storage)
aws ec2 stop-instances --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE

# Verify instances are stopped
aws ec2 describe-instances \
  --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE \
  --query 'Reservations[].Instances[][InstanceId,State.Name]' \
  --output table
```

Start instances when needed:

```
# Start instances
aws ec2 start-instances --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE

# Wait for instances to be running
aws ec2 wait instance-running --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE

# Get new public IPs (they change after stop/start)
aws ec2 describe-instances \
  --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE \
  --query 'Reservations[].Instances[][Tags[?Key==`Name`].Value|[0],PublicIpAddress]' \
  --output table
```

Step 7.2: Complete Cleanup (Destroy Lab)

⚠ **WARNING:** This will permanently delete all resources!


```
# Terminate all EC2 instances
aws ec2 terminate-instances --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE

# Wait for instances to terminate
aws ec2 wait instance-terminated --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE

# Delete security groups
aws ec2 delete-security-group --group-id $SG_MAIN
aws ec2 delete-security-group --group-id $SG_SECURITY_TOOLS

# Detach and delete Internet Gateway
aws ec2 detach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id $VPC_ID
aws ec2 delete-internet-gateway --internet-gateway-id $IGW_ID

# Delete subnet
aws ec2 delete-subnet --subnet-id $SUBNET_ID

# Delete VPC
aws ec2 delete-vpc --vpc-id $VPC_ID

# Delete key pair
aws ec2 delete-key-pair --key-name redblue-lab-keypair
rm ~/.ssh/redblue-lab-keypair.pem

echo "Lab cleanup complete!"
```

Step 7.3: Verify All Resources Deleted

```
# Check for any remaining instances
aws ec2 describe-instances --query 'Reservations[].Instances[][InstanceId,State.Name,Tags[?Key==`Name`.Value][0]]' --output table

# Check for VPCs
aws ec2 describe-vpcs --query 'Vpcs[][VpcId,Tags[?Key==`Name`.Value][0]]' --output table

# Check for security groups
aws ec2 describe-security-groups --query 'SecurityGroups[][GroupId,GroupName]' --output table

# Review AWS billing dashboard
# https://console.aws.amazon.com/billing/home
```

Resources & References

Official Documentation

AWS Services:

- AWS CLI Documentation: <https://docs.aws.amazon.com/cli/>
- Amazon EC2 User Guide: <https://docs.aws.amazon.com/ec2/>
- Amazon VPC User Guide: <https://docs.aws.amazon.com/vpc/>
- AWS Free Tier: <https://aws.amazon.com/free/>

Security Tools:

- Kali Linux Documentation: <https://www.kali.org/docs/>
- Splunk Enterprise Documentation: <https://docs.splunk.com/Documentation/Splunk>
- Tenable Nessus Documentation: <https://docs.tenable.com/nessus/>
- Metasploit Documentation: <https://docs.metasploit.com/>

GitHub Repositories

- Cloud Cybersecurity Homelab: <https://github.com/collinsmc23/cloud-cybersecurity-homelab>
- Cloud-Hosted Cybersecurity Homelab: <https://github.com/ma1ha/Cloud-Hosted-Cybersecurity-Homelab>
- AWS Security Projects: <https://github.com/0xrajneesh/Cloud-Security-Projects-For-Beginners>

Video Tutorials

- Build Cloud Red/Blue Team Lab: <https://www.youtube.com/watch?v=2cMkpLoKUj0>
- Kali Linux on AWS EC2: https://www.youtube.com/watch?v=Jnx7g_5Kn6s
- Security Onion Setup: <https://www.youtube.com/watch?v=8BwDtKGsB2A>
- AWS CLI Linux Tutorial: <https://www.youtube.com/watch?v=1OqMQPx8Jno>

Download Links

AWS Tools:

- AWS CLI: <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
- AWS Console: <https://console.aws.amazon.com/>

Security Tools:

- Splunk Enterprise: https://www.splunk.com/en_us/download/splunk-enterprise.html
- Splunk Universal Forwarder: https://www.splunk.com/en_us/download/universal-forwarder.html
- Tenable Nessus Essentials: <https://www.tenable.com/products/nessus/nessus-essentials>
- Remmina RDP Client: <https://remmina.org/how-to-install-remmina/>

AWS Marketplace AMIs

- Kali Linux: <https://aws.amazon.com/marketplace> (search "Kali Linux")
- Netspectrum Ubuntu Desktop: <https://aws.amazon.com/marketplace> (search "Netspectrum Ubuntu")
- Windows Server 2022: Available in Quick Start AMIs

Community Resources

Forums & Communities:

- Kali Linux Forums: <https://forums.kali.org/>
- Splunk Community: <https://community.splunk.com/>
- AWS Forums: <https://forums.aws.amazon.com/>
- Reddit r/cybersecurity: <https://reddit.com/r/cybersecurity>
- Reddit r/aws: <https://reddit.com/r/aws>

Learning Platforms:

- TryHackMe: <https://tryhackme.com/>
- HackTheBox: <https://www.hackthebox.com/>
- AWS Skill Builder: <https://skillbuilder.aws/>
- Cybrary: <https://www.cybrary.it/>

Cost Estimation Tools

- AWS Pricing Calculator: <https://calculator.aws/>
- EC2 Pricing: <https://aws.amazon.com/ec2/pricing/>
- VPC Pricing: <https://aws.amazon.com/vpc/pricing/>

Best Practices Guides

- AWS Security Best Practices: <https://aws.amazon.com/security/security-resources/>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>
- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- CIS Benchmarks: <https://www.cisecurity.org/cis-benchmarks/>

Troubleshooting Common Issues

Cannot SSH into Kali/Ubuntu Instance

Issue: Connection timeout or refused.

Solutions:

```
# 1. Verify instance is running
aws ec2 describe-instances --instance-ids $KALI_INSTANCE --query 'Reservations[0].Instances[0].State.Name'

# 2. Check security group allows SSH from your IP
MY_IP=$(curl -s https://checkip.amazonaws.com)
echo "Your IP: $MY_IP/32"

# 3. Verify key permissions
ls -l ~/.ssh/redblue-lab-keypair.pem
# Should show: -r----- (400 permissions)

# 4. Wait 2-3 minutes after instance launch for SSH to be ready

# 5. Use verbose SSH for debugging
ssh -v -i ~/.ssh/redblue-lab-keypair.pem kali@$KALI_IP
```

Cannot RDP into Windows Instance

Issue: RDP connection fails.

Solutions:

1. Verify instance has been running for at least 5 minutes (Windows takes time to boot)
2. Check security group allows port 3389 from your IP
3. Ensure you're using correct password (decrypt with private key)
4. Try connecting using private IP from Kali instance
5. Check Windows Firewall is disabled for RDP

Splunk Not Receiving Windows Logs

Issue: No data in `index=win-security`.

Solutions:

```
# 1. Verify Splunk receiver is running
# On Security Tools box:
netstat -tulpn | grep 9997

# 2. Check Security Group allows port 9997
aws ec2 describe-security-groups --group-ids $SG_SECURITY_TOOLS

# 3. Verify Universal Forwarder is running on Windows
# On Windows: sc query SplunkForwarder

# 4. Check inputs.conf exists and is correctly formatted
# File: C:\Program Files\SplunkUniversalForwarder\etc\system\local\inputs.conf

# 5. Restart Universal Forwarder
# On Windows: cd "C:\Program Files\SplunkUniversalForwarder\bin"
# splunk.exe restart

# 6. Check Splunk forwarder logs
# C:\Program Files\SplunkUniversalForwarder\var\log\splunk\splunkd.log
```

High AWS Costs

Issue: Unexpected charges.

Solutions:

```
# Stop instances when not in use
aws ec2 stop-instances --instance-ids $KALI_INSTANCE $WINDOWS_INSTANCE $SECURITY_INSTANCE

# Use smaller instance types
# Kali: t2.micro instead of t2.medium
# Security Tools: t2.large instead of t3.large

# Delete unused EBS volumes
aws ec2 describe-volumes --filters Name=status,Values=available

# Set up billing alerts in AWS Console
# Billing → Billing preferences → Receive Billing Alerts

# Use AWS Cost Explorer to track spending
# https://console.aws.amazon.com/cost-management/home
```

Success Checklist

Infrastructure Setup

- ☐ AWS CLI installed and configured on Parrot OS
- ☐ IAM user created with EC2/VPC permissions
- ☐ SSH key pair created and stored securely
- ☐ Custom VPC created (10.0.0.0/16)
- ☐ Public subnet created (10.0.1.0/24)
- ☐ Internet Gateway attached to VPC
- ☐ Route table configured with default route to IGW
- ☐ Security groups created with proper rules

Instance Deployment

- ☐ Kali Linux instance launched and accessible via SSH
- ☐ Windows Server instance launched and accessible via RDP
- ☐ Ubuntu Security Tools instance launched
- ☐ All instances have public IPs assigned
- ☐ All instances can ping each other

Tool Installation

- ☐ XRDP installed on Kali (optional for GUI)
- ☐ Splunk Enterprise installed on Security Tools box
- ☐ Splunk receiving port 9997 configured
- ☐ Splunk index win-security created
- ☐ Splunk Universal Forwarder installed on Windows
- ☐ Windows logs appearing in Splunk
- ☐ Nessus Essentials installed and activated
- ☐ Metasploit and tools updated on Kali

Testing & Validation

- ☐ Network scans from Kali to Windows successful
- ☐ RDP from Kali to Windows works
- ☐ Windows security events visible in Splunk
- ☐ Nessus scan completed against Windows target
- ☐ Traffic capture working with tcpdump
- ☐ Attack simulations generating alerts in Splunk

Documentation & Management

- ☐ Instance IPs and credentials documented
- ☐ Billing alerts configured in AWS
- ☐ Understanding of start/stop procedures for cost savings
- ☐ Cleanup procedure tested

Next Steps & Advanced Projects

Expand Your Lab

1. **Add More Target Systems:**
 - Deploy Metasploitable2 or DVWA for web app testing
 - Add Linux vulnerable targets (Ubuntu with outdated packages)
 - Create Active Directory domain controller
2. **Implement Traffic Mirroring:**
 - Configure VPC Traffic Mirroring for deeper packet inspection
 - Mirror traffic from Windows to Security Tools box
 - Analyze with Wireshark or Security Onion
3. **Automate with Terraform:**
 - Create Terraform configs for reproducible deployments
 - Version control your infrastructure
 - One-command lab creation and destruction
4. **Advanced Monitoring:**
 - Deploy Security Onion alongside Splunk
 - Configure ELK stack (Elasticsearch, Logstash, Kibana)
 - Implement Suricata IDS/IPS
 - Set up Wazuh HIDS
5. **Integrate AWS Security Services:**
 - Enable AWS GuardDuty for threat detection
 - Configure AWS Security Hub
 - Implement AWS CloudTrail logging
 - Use AWS Config for compliance

Skill Development

1. **Red Team Skills:**
 - Practice with Metasploit modules
 - Learn PowerShell exploitation
 - Study Active Directory attacks
 - Practice privilege escalation
2. **Blue Team Skills:**
 - Create Splunk correlation searches
 - Build detection rules for common attacks
 - Practice incident response procedures
 - Develop threat hunting queries
3. **Certifications to Pursue:**
 - AWS Certified Security - Specialty
 - CompTIA Security+
 - CompTIA CySA+
 - Certified Ethical Hacker (CEH)
 - Offensive Security Certified Professional (OSCP)

Legal & Ethical Disclaimer

IMPORTANT: This lab is designed for **educational and authorized testing purposes only**.

Ethical Guidelines

1. **Only test systems you own or have explicit written permission to test**
2. **Never use these techniques against production systems without authorization**
3. **Comply with AWS Acceptable Use Policy:** <https://aws.amazon.com/aup/>
4. **Follow responsible disclosure practices if you discover vulnerabilities**
5. **Respect privacy and data protection laws (GDPR, CCPA, etc.)**

AWS Penetration Testing Policy

AWS **allows** penetration testing of your own resources, but you must:

- Only test your own AWS resources
- Not perform DoS or DDoS attacks
- Follow AWS Customer Support Policy for Penetration Testing

- Review: <https://aws.amazon.com/security/penetration-testing/>

Legal Considerations

Unauthorized access to computer systems is illegal under laws including:

- Computer Fraud and Abuse Act (CFAA) - United States
- Computer Misuse Act - United Kingdom
- Various international cybercrime laws

Always obtain proper authorization before conducting security testing.

Conclusion

Congratulations! You've successfully built a comprehensive AWS Red Team vs Blue Team cybersecurity homelab. This environment provides hands-on experience with:

- **Cloud infrastructure** design and deployment
- **Offensive security** techniques and tools
- **Defensive monitoring** with industry-standard SIEM
- **Vulnerability assessment** and remediation
- **Network security** concepts and protocols

What You've Accomplished

- ✓ Built isolated cloud security testing environment
- ✓ Deployed and configured Kali Linux for penetration testing
- ✓ Set up Windows target system with monitoring
- ✓ Implemented Splunk for log aggregation and analysis
- ✓ Configured Nessus for vulnerability scanning
- ✓ Practiced attack simulation and detection
- ✓ Learned AWS security best practices

Continue Your Journey

This lab is your **playground for security research**. Continue exploring, learning, and most importantly, **practice responsibly**.

Remember to **stop your instances** when not in use to control costs, and always follow ethical hacking principles.

Happy Hacking! ☒✕

Document Version: 1.0

Last Updated: November 6, 2025

Created for: Parrot OS / Linux Users

Lab Difficulty: Intermediate
